Take Home Assignment
FileAI

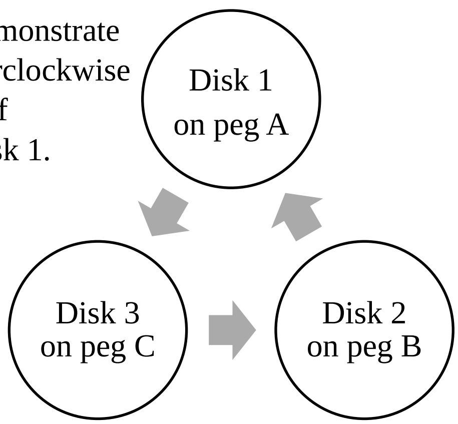
Tower of Hanoi

The Tower of Hanoi is a mathematical puzzle invented by the French mathematician Édouard Lucas in 1883. There is a story about an Indian temple in Kashi Vishwanath which contains a large room with three time-worn posts in it, surrounded by 64 golden disks. Brahmin priests, acting out the command of an ancient prophecy, have been moving these disks in accordance with the immutable rules of Brahma since that time. The puzzle configuration and rules are as follows:

- There are *n* disks and three pegs: A, B, C. The *n* disks are different in size, and all disks are initially placed on peg A that their sizes increase from top to bottom.
- Only one disk can be moved at a time, from the top of a peg to an empty peg or to a peg with larger disk than itself on top.
- The goal is to move all *n* disks from peg A to either peg B or peg C – this is the goal of the original tower of Hanoi puzzle, but our goal in this question is slightly different.

It is easy to prove that **the minimum number of moves** needed to complete the puzzle with *n* disks is 2^n-1 using recursion. Given a state with all disks on peg A, A simple algorithm to move all disks to another peg with the minimum number of moves is to move the smallest disk (disk 1: D1) on odd moves (the 1st, 3rd, 5th… moves) in a circular manner **ACBACB…** (counterclockwise, as shown left); and make the only possible move which does not involve disk 1 on even moves (the 2nd, 4th, 6th… moves). For example, with 3 disks initially placed on peg A, 5 moves will lead to a state of having D1 on A, D2 on B and D3 on C as shown left. According to the simple algorithm above, the 5 moves are: (D1:A→C), (D2:A→B), (D1:C→B), (D3:A→C) and (D1:B→A) where D1 is the smallest disk, D3 is the largest disk and (D2:A→B) means disk 2 (the middle disk) is moved from peg A to peg B.

Arrows demonstrate the counterclockwise direction of moving disk 1.



Given a state of *n* disks placed on pegs A, B and C with their sizes increasing from top to bottom, please write a program to determine which peg the disks were initially placed, and how many moves there were to reach this state.

Test inputs begin with a number indicating the number of test cases below, and each test case is described by a line describing the state with disks on pegs A, B, C separated by commas, and then separated by spaces between disks on the same peg in ascending order. For each test case, output (i) the initial peg, and (ii) the number of moves. Output "impossible" if no matter which peg to start with, the described state is non-reachable. For the special case that all disks are on the same peg, as this state can be an initial state or a final state, please take it as an initial state. The number of moves should be between 0 and 2^n-2 both inclusive.

Sample code is provided in the file **A1Q2.py**. You need to modify the function `tower_hanoi`. The total number of disks is between 3 and 64.

There will be 6 testcases with 1 mark each, 2 additional marks are awarded if your code is implemented using recursion given that your code passes all 6 testcases. The remaining 2 marks are awarded with the correct justification of the time complexity of your algorithm.

Sample Input

```
10
 1, 2, 3
 1, 3, 2
 3, 2, 1
 2, , 1 3
 4, 1 2, 3
 2 5, 3 4, 1
 1 6 9 10, 2 7 8 11, 3 4 5 12 13
 2 5 6 9 12 13 14 15, 3 10 11, 1 4 7 8
 6 7 12 15, 1 2 5 8 9 10 13 16, 3 4 11 14 17 18
 2 7 10 17 18, 3 6 9 12 13 16 19, 1 4 5 8 11 14 15 20
```

Sample Output

```
A 5
B 2
A 2
impossible
A 4
A 13
C 1245
B 31029
B 218220
C 301733
```

Running python skeleton with sample input:

1. Open "Anaconda Prompt"
2. Go to the directory where you put the file **A1Q1.py** and **A1Q1.in**, using command **cd**
3. Run command **python A1Q1.py < A1Q1.in**
4. You may want to create a test input called **my_own_test.in** to design a test case for your own program, the command would then be **python A1Q1.py < my_own_test.in**
5. Same applies to Question 2, so you may run **python A1Q2.py < A1Q2.in**

Note There's only one Question in this assignmnet use the appropriate names for for the .py and .in file when testing.